In [21]:
```
import pandas as pd
import numpy as np
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, classification_report
from sklearn.preprocessing import LabelEncoder
```

In [22]:
```
df = pd.read_csv("/kaggle/input/competitions/fraud-detection-cpe-232-data-models/train.csv")
```

EDA

In [23]:
```
df
```

Out[23]:

	id	time_ind	transac_type	amount	src_acc	src_bal	src_new_bal	dst_acc	dst_bal	dst_new_bal	is_fraud	is_flagged_fraud
0	0	355	CASH_OUT	56964.57	acc4649128	20090.00	0.00	acc8350663	0.00	56964.57	0	0
1	1	284	CASH_OUT	108454.37	acc6660911	505.00	0.00	acc1547749	0.00	108454.37	0	0
2	2	378	NaN	467324.33	acc7810035	1634.00	0.00	acc3033156	258071.61	725395.95	0	0
3	3	129	CASH_IN	274154.97	acc4833344	3171570.20	3445725.17	acc5071405	1692712.58	987166.60	0	0
4	4	217	PAYMENT	NaN	acc8839024	23231.09	0.00	acc6776502	0.00	0.00	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
5408222	5408222	153	PAYMENT	1895.99	acc7005579	0.00	0.00	acc5686902	0.00	0.00	0	0
5408223	5408223	402	CASH_OUT	347110.99	acc4716956	103785.00	0.00	acc660558	87871.75	434982.74	0	0
5408224	5408224	304	PAYMENT	13259.63	acc4771603	0.00	0.00	acc7665142	0.00	0.00	0	0
5408225	5408225	298	PAYMENT	24122.92	acc5114700	0.00	0.00	acc2715964	0.00	0.00	0	0
5408226	5408226	154	PAYMENT	6865.63	acc5526779	0.00	0.00	acc1347140	0.00	0.00	0	0

5408227 rows × 12 columns

In [24]:
```
df.isnull().sum()
```

Out[24]:
```
id                    0
time_ind              0
transac_type     648331
amount           702632
src_acc               0
src_bal          269799
src_new_bal           0
dst_acc               0
dst_bal          270191
dst_new_bal           0
is_fraud              0
is_flagged_fraud      0
dtype: int64
```

In [25]:
```
df.groupby('transac_type').agg({"is_fraud":"sum"})
```

Out[25]:

transac_type	is_fraud
CASH_IN	0
CASH_OUT	2789
DEBIT	0
PAYMENT	0
TRANSFER	2731

In [26]:
```
df['transac_type'] = df['transac_type'].fillna("UNKNOWN")
```

In [27]:
```
df.groupby('transac_type').agg({"is_fraud":"sum"})
```

Out[27]:

transac_type	is_fraud
CASH_IN	0
CASH_OUT	2789
DEBIT	0
PAYMENT	0
TRANSFER	2731
UNKNOWN	1461

In [28]:
```
df_complete = df.dropna()
```

In [29]:
```
df.head()
```

Out[29]:

	id	time_ind	transac_type	amount	src_acc	src_bal	src_new_bal	dst_acc	dst_bal	dst_new_bal	is_fraud	is_flagged_fraud
0	0	355	CASH_OUT	56964.57	acc4649128	20090.00	0.00	acc8350663	0.00	56964.57	0	0
1	1	284	CASH_OUT	108454.37	acc6660911	505.00	0.00	acc1547749	0.00	108454.37	0	0
2	2	378	UNKNOWN	467324.33	acc7810035	1634.00	0.00	acc3033156	258071.61	725395.95	0	0
3	3	129	CASH_IN	274154.97	acc4833344	3171570.20	3445725.17	acc5071405	1692712.58	987166.60	0	0
4	4	217	PAYMENT	NaN	acc8839024	23231.09	0.00	acc6776502	0.00	0.00	0	0

row ที่ 2(id=1) CASH_OUT 108454.37 แต่ src_bal จาก 1634 -> 0.00 เห็นว่าจำนวนเงินเปลี่ยนแปลงไม่เท่า amount

In [30]:
```
len(df_complete)
```

Out[30]: 4246857

In [31]:
```
len(df_complete[abs(df_complete['src_new_bal']-df_complete['src_bal'])!=df_complete['amount']])
```

Out[31]: 3619991

In [32]:
```
df.groupby('transac_type').agg({"amount":"mean"})
```

Out[32]:

transac_type	amount
CASH_IN	157324.794133
CASH_OUT	162066.391013
DEBIT	5498.528771
PAYMENT	13057.828880
TRANSFER	709435.062306
UNKNOWN	249463.589484

In [33]:
```
df["time_ind"].value_counts()
```

Out[33]:
```
time_ind
19     43651
18     42253
187    41861
235    40279
307    39686
       ...
480        3
54         3
113        2
662        1
112        1
Name: count, Length: 743, dtype: int64
```

In [34]:
```
df[df['amount'] == df['src_bal']]['is_fraud'].value_counts()
```

Out[34]:
```
is_fraud
1    4867
Name: count, dtype: int64
```

In [35]:
```
df[df['src_new_bal'] == 0]['is_fraud'].value_counts()
```

Out[35]:
```
is_fraud
0    3061345
1       6841
Name: count, dtype: int64
```

Feature Engineering

In [36]:
```
def FE(df):
    #transac_type imputation and Filter only fraud type
    df['transac_type'] = df['transac_type'].fillna("UNKNOWN")
    df = df[df['transac_type'].isin(['TRANSFER', 'CASH_OUT', 'UNKNOWN'])].copy()

    #Add data missing flag
    df['is_missing_amt'] = df['amount'].isnull().astype(int)
    df['is_missing_src'] = df['src_bal'].isnull().astype(int)
    df['is_missing_dst'] = df['dst_bal'].isnull().astype(int)

    #amt column imputation
    mask = df['amount'].isna() & df['src_bal'].notna() & df['src_new_bal'].notna()
    df.loc[mask, 'amount'] = abs(df.loc[mask, 'src_bal'] - df.loc[mask, 'src_new_bal'])

    #Compare amt and src/dst_new
    df["amt_src_cmp"] = (df['amount'] > df['src_bal']).astype(int)
    df["amt_dst_new_cmp"] = (df['amount'] > df['dst_new_bal']).astype(int)

    #ratio of amt and changing of src
    df['actual_src_diff'] = abs(df['src_bal'] - df['src_new_bal'])
    df['amt_src_diff_ratio'] = df['amount'] / (df['actual_src_diff'] + 1e-9)

    #ratio of amt and changing of dst
    df['actual_dst_diff'] = abs(df['dst_new_bal'] - df['dst_bal'])
    df['amt_dst_diff_ratio'] = df['amount'] / (df['actual_dst_diff'] + 1e-9)
    # คนที่โอนออกหมดบัญชี
    df['is_perfect_drain'] = ((df['amount'] == df['src_bal']) & (df['src_new_bal'] == 0)).astype(int)
    # ratio of amt and src
    df['amt_src_ratio'] = df['amount'] / (df['src_bal'] + 1e-9)


    #time_ind density
    df['tx_density'] = df.groupby('time_ind')['time_ind'].transform('count')

    #fill na
    df['src_bal'] = df['src_bal'].fillna(-1)
    df['dst_bal'] = df['dst_bal'].fillna(-1)
    df['amount'] = df['amount'].fillna(-1)
    #One-hot encoding
    df = pd.get_dummies(df, columns=['transac_type'], drop_first=False)

    return df
```

In [37]:
```
df = FE(df)
```

Model Training

In [38]:
```
features = [
    'amount', 'src_bal', 'src_new_bal','dst_bal', 'dst_new_bal' ,
    'is_perfect_drain' , 'amt_src_ratio', 'tx_density',
    'amt_src_cmp' , 'amt_dst_new_cmp','amt_src_diff_ratio','amt_dst_diff_ratio',
```

```
    'is_missing_amt' , 'transac_type_CASH_OUT', 'transac_type_TRANSFER', 'transac_type_UNKNOWN'
]
```

In [39]:
```
X = df[features]
y = df['is_fraud']
```

In [40]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

ratio = (y == 0).sum() / (y == 1).sum()

xgb_model = XGBClassifier(
    device="cuda",
    n_estimators=1000,
    max_depth= 6,
    learning_rate=0.03,
    scale_pos_weight=ratio,
    eval_metric='aucpr',
    tree_method='hist',
    random_state=42,
    n_jobs=-1
)

xgb_model.fit(X_train, y_train,eval_set=[(X_train, y_train), (X_test, y_test)], verbose=50   )
```

```
[0]     validation_0-aucpr:0.73364      validation_1-aucpr:0.73399
[50]    validation_0-aucpr:0.82474      validation_1-aucpr:0.82034
[100]   validation_0-aucpr:0.97279      validation_1-aucpr:0.97223
[150]   validation_0-aucpr:0.98984      validation_1-aucpr:0.98856
[200]   validation_0-aucpr:0.99139      validation_1-aucpr:0.98931
[250]   validation_0-aucpr:0.99415      validation_1-aucpr:0.99017
[300]   validation_0-aucpr:0.99592      validation_1-aucpr:0.99041
[350]   validation_0-aucpr:0.99716      validation_1-aucpr:0.99046
[400]   validation_0-aucpr:0.99813      validation_1-aucpr:0.99062
[450]   validation_0-aucpr:0.99873      validation_1-aucpr:0.99079
[500]   validation_0-aucpr:0.99911      validation_1-aucpr:0.99080
[550]   validation_0-aucpr:0.99939      validation_1-aucpr:0.99070
[600]   validation_0-aucpr:0.99958      validation_1-aucpr:0.99075
[650]   validation_0-aucpr:0.99970      validation_1-aucpr:0.99084
[700]   validation_0-aucpr:0.99978      validation_1-aucpr:0.99095
[750]   validation_0-aucpr:0.99982      validation_1-aucpr:0.99100
[800]   validation_0-aucpr:0.99985      validation_1-aucpr:0.99093
[850]   validation_0-aucpr:0.99987      validation_1-aucpr:0.99094
[900]   validation_0-aucpr:0.99989      validation_1-aucpr:0.99095
[950]   validation_0-aucpr:0.99991      validation_1-aucpr:0.99102
[999]   validation_0-aucpr:0.99992      validation_1-aucpr:0.99104
```

Out[40]:
```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device='cuda', early_stopping_rounds=None,
              enable_categorical=False, eval_metric='aucpr', feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.03, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
              max_leaves=None, min_child_weight=None, missing=nan,
```

In [41]:
```
#threshold tuning
y_probs = xgb_model.predict_proba(X_test)[:, 1]
best_f1, best_t = 0, 0
for t in np.arange(0.90, 0.999, 0.001):
    f = f1_score(y_test, (y_probs >= t).astype(int))
    if f > best_f1:
        best_f1, best_t = f, t

print(f"\nBest Validation F1-Score: {best_f1:.4f} at Threshold: {best_t:.3f}")
```

```
/usr/local/lib/python3.12/dist-packages/xgboost/core.py:774: UserWarning: [09:12:35] WARNING: /workspace/src/common/error_msg.cc:62: Falling back to prediction using DMatrix due to mism
atched devices. This might lead to higher memory usage and slower performance. XGBoost is running on: cuda:0, while the input data is on: cpu.
Potential solutions:
- Use a data structure that matches the device ordinal in the booster.
- Set the device for booster before call to inplace_predict.

This warning will only be shown once.

  return func(**kwargs)
Best Validation F1-Score: 0.9866 at Threshold: 0.990
```

In [42]:
```
#ดู case ที่ทายผิด
val_results = X_test.copy()
val_results['actual'] = y_test
val_results['prob'] = xgb_model.predict_proba(X_test)[:, 1]
val_results['pred'] = (val_results['prob'] >= 0.962).astype(int)

fn_cases = val_results[(val_results['actual'] == 1) & (val_results['pred'] == 0)].sort_values('prob', ascending=False)

fp_cases = val_results[(val_results['actual'] == 0) & (val_results['pred'] == 1)].sort_values('prob', ascending=True)

print(f"FN: {len(fn_cases)} เคส")
print(f"FP: {len(fp_cases)} เคส")
```

```
FN: 28 เคส
FP: 13 เคส
```

In [43]:
```
fn_cases
```

Out[43]:

	amount	src_bal	src_new_bal	dst_bal	dst_new_bal	is_perfect_drain	amt_src_ratio	tx_density	amt_src_cmp	amt_dst_new_cmp	amt_src_diff_ratio	amt_dst_diff_ratio	is_missing_amt	t
2477315	539658.73	-1.00	0.0	0.00	539658.73	0	NaN	52	0	0	NaN	1.000000	0	
2565550	21574.55	-1.00	0.0	0.00	21574.55	0	NaN	6	0	0	NaN	1.000000	0	
355205	127091.33	-1.00	0.0	656699.62	783790.95	0	NaN	15	0	0	NaN	1.000000	0	
3834568	29174.80	29174.80	0.0	-1.00	0.00	1	1.000000	11652	0	1	1.000000	NaN	1	
1015372	125146.75	-1.00	0.0	1820417.69	1945564.44	0	NaN	4	0	0	NaN	1.000000	0	
2610139	157277.00	-1.00	0.0	369445.83	526722.83	0	NaN	13	0	0	NaN	1.000000	0	
2263801	67128.33	-1.00	0.0	623880.13	691008.45	0	NaN	3	0	0	NaN	1.000000	0	
451400	7442005.19	-1.00	0.0	0.00	7442005.19	0	NaN	3346	0	0	NaN	1.000000	0	
2944745	-1.00	-1.00	0.0	0.00	805319.81	0	NaN	11731	0	0	NaN	NaN	1	
1611915	1076739.91	-1.00	0.0	2186542.64	3263282.55	0	NaN	9	0	0	NaN	1.000000	0	
3155380	-1.00	-1.00	0.0	160374.92	3832228.78	0	NaN	16377	0	0	NaN	NaN	1	
4100035	4354175.33	-1.00	0.0	-1.00	4778329.47	0	NaN	13559	0	0	NaN	NaN	0	
2777	8607089.62	-1.00	0.0	457121.57	9064211.19	0	NaN	19638	0	0	NaN	1.000000	0	
1911304	81244.31	-1.00	0.0	62279.28	143523.59	0	NaN	16119	0	0	NaN	1.000000	0	
796528	235075.62	235075.62	0.0	436894.35	310170.72	1	1.000000	7639	0	0	1.000000	1.855026	1	
3229250	-1.00	-1.00	0.0	0.00	0.00	0	NaN	1633	0	0	NaN	NaN	1	
5315041	360688.30	360688.30	0.0	-1.00	360688.30	1	1.000000	13942	0	0	1.000000	NaN	1	
3816658	696763.08	696763.08	0.0	366114.58	1347073.81	1	1.000000	20017	0	0	1.000000	0.710288	1	
4781151	-1.00	-1.00	0.0	-1.00	461867.40	0	NaN	1069	0	0	NaN	NaN	1	
1707681	450489.27	-1.00	0.0	122047.64	572536.91	0	NaN	15486	0	0	NaN	1.000000	0	
2335754	-1.00	-1.00	0.0	7519338.35	9114925.81	0	NaN	3945	0	0	NaN	NaN	1	
1446420	-1.00	-1.00	0.0	1219255.67	1225340.97	0	NaN	3945	0	0	NaN	NaN	1	
1830701	131594.12	-1.00	0.0	0.00	131594.12	0	NaN	8011	0	0	NaN	1.000000	0	
5268608	432958.83	-1.00	0.0	0.00	432958.83	0	NaN	8079	0	0	NaN	1.000000	0	
894660	282018.00	-1.00	0.0	2255136.89	2537154.89	0	NaN	14780	0	0	NaN	1.000000	0	
781313	2806.00	2806.00	0.0	26202.00	0.00	1	1.000000	736	0	1	1.000000	0.107091	1	
5277241	132842.64	4499.08	0.0	0.00	132842.64	0	29.526623	736	1	0	29.526623	1.000000	0	
2587737	0.00	0.00	0.0	267522.87	267522.87	1	0.000000	21	0	0	0.000000	0.000000	1	

In [44]:
```
fp_cases
```

Out[44]:

	amount	src_bal	src_new_bal	dst_bal	dst_new_bal	is_perfect_drain	amt_src_ratio	tx_density	amt_src_cmp	amt_dst_new_cmp	amt_src_diff_ratio	amt_dst_diff_ratio	is_missing_amt	tr
2463290	849703.90	-1.00	0.0	39583.17	889287.07	0	NaN	867	0	0	NaN	1.000000e+00	0	
1093276	-1.00	-1.00	0.0	4286.23	1878150.67	0	NaN	8081	0	0	NaN	NaN	1	
3828545	-1.00	-1.00	0.0	92688.90	3754431.42	0	NaN	3552	0	0	NaN	NaN	1	
1673197	-1.00	-1.00	0.0	378986.54	589763.66	0	NaN	149	0	0	NaN	NaN	1	
4991506	-1.00	-1.00	0.0	699351.89	937530.54	0	NaN	148	0	0	NaN	NaN	1	
4776746	-1.00	-1.00	0.0	591005.83	1073220.42	0	NaN	18	0	0	NaN	NaN	1	
4066964	-1.00	-1.00	0.0	0.00	3692821.64	0	NaN	10691	0	0	NaN	NaN	1	
4047513	529175.59	529175.59	0.0	1908803.79	2265853.97	1	1.0	2017	0	0	1.0	1.482076e+00	1	
4616883	-1.00	-1.00	0.0	556861.67	733612.15	0	NaN	109	0	0	NaN	NaN	1	
462450	450663.00	450663.00	0.0	-1.00	435891.41	1	1.0	15285	0	1	1.0	NaN	1	
4352977	4719.00	4719.00	0.0	0.00	0.00	1	1.0	8081	0	1	1.0	4.719000e+12	1	
607586	-1.00	-1.00	0.0	0.00	0.00	0	NaN	122	0	0	NaN	NaN	1	
1269287	84606.00	84606.00	0.0	0.00	0.00	1	1.0	20017	0	1	1.0	8.460600e+13	1	

Predict submission

In [45]:
```
test_df = pd.read_csv("/kaggle/input/competitions/fraud-detection-cpe-232-data-models/test.csv")
sample_sub = pd.read_csv("/kaggle/input/competitions/fraud-detection-cpe-232-data-models/sample_submission.csv")

test_processed = FE(test_df)

y_test_probs = xgb_model.predict_proba(test_processed[features])[:, 1]
test_processed['is_fraud'] = (y_test_probs > best_t).astype(int)

final_sub = sample_sub[['id']].merge(test_processed[['id', 'is_fraud']], on='id', how='left')
final_sub['is_fraud'] = final_sub['is_fraud'].fillna(0).astype(int)

final_sub.to_csv('submission.csv', index=False)
print("Submission file created.")
```

```
Submission file created.
```

In [46]:
```
final_sub["is_fraud"].value_counts()
```

Out[46]:
```
is_fraud
0    953147
1      1246
Name: count, dtype: int64
```

Feature Importance

In [47]:
```
#Feature importance by gain
importance_gain = xgb_model.get_booster().get_score(importance_type='gain')

fi_df = pd.DataFrame({
    'Feature': list(importance_gain.keys()),
    'Gain': list(importance_gain.values())
})

fi_df.sort_values(by='Gain', ascending=False)
```

Out[47]:

	Feature	Gain
5	is_perfect_drain	44111.148438
8	amt_src_cmp	4958.255371
11	amt_dst_diff_ratio	4795.467773
12	is_missing_amt	1486.428589
10	amt_src_diff_ratio	923.642761
1	src_bal	810.191406
7	tx_density	801.941956
2	src_new_bal	372.422638
15	transac_type_UNKNOWN	370.890198
6	amt_src_ratio	236.099319
14	transac_type_TRANSFER	232.245880
3	dst_bal	227.048935
0	amount	173.431458
9	amt_dst_new_cmp	169.732208
4	dst_new_bal	142.058273
13	transac_type_CASH_OUT	103.381401

In [ ]:
```
```